RL-Based Smart Parking Allocation

AI Assignment 2 — Real-Life Use Cases of Reinforcement Learning

Team: Duggirala Manoj (B24CM1022, b24cm1022@iitj.ac.in) and Likith Reddy (B24CM1069, b24cm1069@iitj.ac.in)
GitHub: Add your repository URL before submission.

1. Problem and objective

Parking allocation becomes difficult when vehicles arrive continuously, spaces have different walking distances, and parked vehicles leave at uncertain times. A simple fixed rule can overload one zone while other zones remain available. This project models a small multi-zone car park and trains an RL agent to assign each arriving vehicle to a zone. The objective is to reduce simulated search effort and congestion while keeping walking distance reasonable.

2. Environment

The custom Python simulator has three zones with capacities of 5, 6, and 7 spaces. Their walking-distance costs are 1.8, 1.0, and 3.2 units. One vehicle arrives at each time step. Vehicles have short, medium, or long expected stays; remaining parking times decrease at each step. Demand and stay-type distributions vary across time segments. Each evaluation episode contains 120 vehicle decisions.

3. RL formulation

State S: three zone-occupancy bins (each 0–3), vehicle stay category (0–2), and time/demand segment (0–2). The encoded state has at most $4^3 \times 3 \times 3 = 576$ combinations.
Action A: select one of the three parking zones. Full zones are masked from normal action selection; the environment also defines a penalty for an invalid/full-zone selection.
Transition: the vehicle is assigned, its remaining stay is added to the selected zone, time advances, and all remaining stays decrease by one.

4. Reward function

For a valid assignment, the immediate reward is:
r = 5 − 0.8S − 0.45W − 0.9C − D
where S is simulated search time, W is the zone walking-distance cost, C is a congestion penalty, and D is a small expected-stay blocking cost. Search time increases with pre-assignment occupancy; congestion is penalized above 65% occupancy. An invalid action receives −12. These weights are design choices for the simulator, not real-world currency or measured human preferences.

5. Learning algorithm

We use tabular Q-learning with epsilon-greedy exploration. For state s, action a, reward r, and next state s′, the update is $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]$, with the future-value term omitted at terminal steps. The implementation uses $\alpha = 0.15$, $\gamma = 0.93$, epsilon decay 0.992, and a minimum epsilon of 0.04. Training runs for 1,800 episodes with a fixed seed for reproducibility.

6. Baselines and evaluation

The trained policy is evaluated over 150 seeded episodes and compared with first-available, nearest-available, and least-occupied policies. Metrics include average simulated search time, average walking-distance cost, acceptance rate, congestion penalty, and invalid-action rate. The same episode seeds are used for all policies.

7. Results

Measured mean search-time cost was 1.80 for Q-learning, 3.13 for first-available, 3.24 for nearest-available, and 1.66 for least-occupied. Mean walking-distance costs were 1.91, 1.54, 1.34, and 2.06, respectively. All policies accepted all arrivals in this setup; therefore acceptance rate does not distinguish them. The learned policy trades off search effort against walking distance.

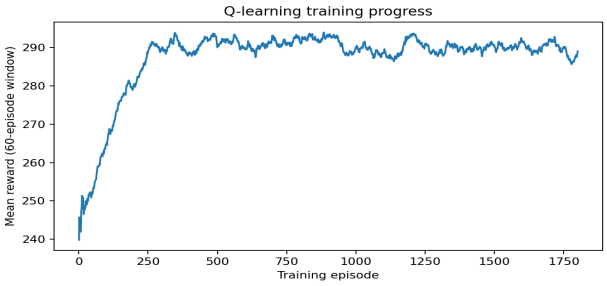


Figure 1. Q-learning reward during training (60-episode moving average).

8. Policy comparison

The bars below show the measured average search-time cost and acceptance rate. The least-occupied baseline has slightly lower search cost in this run, while Q-learning reduces search cost substantially compared with the first-available and nearest-available baselines. This is an important result: the RL agent does not outperform every baseline on every metric, and the experiment should be interpreted as a small simulation study.

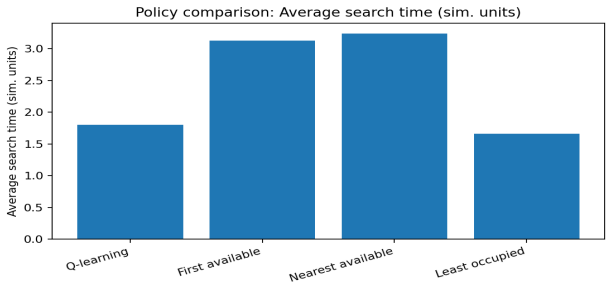


Figure 2. Average simulated search-time cost across policies.

9. Limitations

This is a simplified synthetic environment, not a live parking deployment. It uses stylized vehicle arrivals, expected stays, zone capacities, and hand-designed reward weights. It does not use IIT Jodhpur parking data, model individual vehicle routes, or include real-time sensors. The current evaluation also shows that no policy rejected vehicles, so future work should increase demand pressure and test a wider range of occupancy patterns.

10. Conclusion and next steps

This project provides an original, reproducible RL formulation for parking-zone assignment. It includes a custom simulator, tabular Q-learning, baseline policies, saved learned values, CSV evaluation records, and plots. The results show the trade-off between search effort and walking distance, but they do not establish universal superiority over rule-based methods. Further work should run sensitivity tests over capacities and arrival rates, repeat training over multiple random seeds, and calibrate the simulator using permitted real parking data.

Reproduction

Install Python 3.10+ and the dependency in requirements.txt, then run **python run_experiments.py**. The script trains the Q-table, evaluates all four policies over 150 episodes each, saves CSV files, and regenerates figures. The training seed is 7 and evaluation seeds start at 2000. The repository README contains full setup steps.

Repository contents: parking_env.py, rl_agent.py, run_experiments.py, trained_q_table.pkl, results/*.csv, figures/*.png, requirements.txt, README.md, and this report.

Before submitting: replace “Add your repository URL” with your actual GitHub URL, check the team details, and rerun the script if you change the code or experiment settings.

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

2. Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning, 8*, 279–292. https://doi.org/10.1007/BF00992698

3. The simulator, reward function, and experiment code for this submission were implemented for this project. All reported numerical values are generated by running run_experiments.py with the documented seeds.

4. No external RL repository is used as the implementation base. The cited sources are conceptual references for the RL method.